Software Engineering

Database Integration
SQL, NoSQL

Zhao Zhang
Rutgers University

Topics

- SQL and NoSQL
- Relational Databases
  - ER Model
- Structured Query Language (SQL)
  - Basics
- Database Management Systems (DBMS)
  - MySQL
- Database Access from Programming Languages

What is a Database System?

- **Database**:
  A large collection of related data
  - Shift from *computation* to *information*
- **DBMS** (database management system):
  A set of software programs that controls the organization, storage and retrieval of data from databases
- **Database System**:
  DBMS + data (+ applications)

Unstructured Data Storage

We could use simple text files ...

- Plain text File-1: (each “record” is a new line)

```
"John Doer rented apartment #101 on December 4, 2024"
"Jane Deere rented apartment #103 on January 15, 2025"
...
```

- Plain text File-2:

```
"Tenant John Doer entered apartment #101
   on February 16, 2025 at 5:30 PM"
"Tenant John Doer entered apartment #101
   on February 17, 2025 at 5:48 PM"
...
```

Structured Data Storage

Tenant	Activity	Time	Location
John Doer	Enter	February 16, 2025 at 5:30 PM	apartment #101
	Enter	February 17, 2025 at 5:48 PM	apartment #101
	Rent	December 4, 2024	apartment #101
Jane Deere	Rent	January 15, 2025	apartment #103

JSON—Another Way of Data Structuring

- JSON: JavaScript Object Notation

  https://en.wikipedia.org/wiki/JSON

  - Similar to JavaScript's way of writing objects
  - All property names must be surrounded by double quotes
  - Only simple data expressions are allowed—no function calls, bindings, or anything that involves computation
- Example:

```
{
  "tenant": "John Doer",
  "activity": "enter",
  "time": "February 16, 2025 at 5:30 PM",
  "location": "apartment #101"
}
```

- Most often format used in NoSQL databases

Why Databases?
(instead of plain/unstructured files)

- Abstraction
  - More compact and consistent data
- Query language
  - Data retrieval easier to program and more efficient
    - e.g., (Get time when John Doer entered apartment #101)
- Data integrity when shared between multiple users
- Reliability, Recovery, Security, Data-entry validation
  — all provided by the database system

DBMS: Database Management System

- List of existing Database Systems:

  https://dbdb.io/browse

- Popular SQL-based database:
  MySQL — https://en.wikipedia.org/wiki/MySQL
  - [See the lectures on the REST architecture for an example of using MySQL from NodeJS
- Alternative: NoSQL databases (use JSON)
  — MongoDB, Firebase, …
  - Another option: NeDB … (see next page)

NeDB — a NoQSL Database

- NeDB: a NodeJS library with an API for a NoSQL database (JSON data format)
- NeDB mimics a key subset of MongoDB's API
  - NeDB is fully embedded in the server-side application
    - Does not require an additional server to set up and connect to
    - Unlike MongoDB which is a separate server that you run and make calls from your program
- https://github.com/louischariot/nedb
- https://stackabuse.com/nedb-a-lightweight-javascript-database/

NeDB Example

- Example in NodeJS:

  Insert a JSON “document”,
  then just retrieve the same document using a query and print it to the console

```
const db = new nedb({ filename : "people.db", autoload : true });
db.insert({ firstName : "Billy", lastName : "Joel" },        // insert
   function (inError, inDocument) {
      if (inError) {
         console.log(`Error: ${inError}`);
      } else {
         db.findOne({ firstName : "Billy" },                 // query
            function (inError, inDocument) {
               if (inError) {
                  console.log(`Error: ${inError}`);
               } else {
                  console.log(inDocument);
               }
            }
         };
      }
   }
);
```

Referencing Objects in NoSQL

- NOTE: Unlike SQL, there are no foreign keys in NoSQL.
  Instead, you may consider embedding or references.
- Check more information on the Web, e.g., https://www.mongodb.com/docs/manual/data-modeling/#link-related-data

Database Schema

- **Schema** = the structure of the database
  - e.g., the database consists of information about collections of people and apartments, and the “renting” relationship between them
  - Analogous to type information of a variable in a program
  - **Physical schema**: database design at the physical storage level
  - **Logical schema**: database design at the logical concepts level
- Similar to data types and variables in programming languages

Data Organization

- **Data Model** = a framework for organizing and interpreting data, describes:
  - data
  - data relationships
  - data meaning (semantics)
  - data constraints or business/domain rules
- Entity-Relationship (E-R) model
  - a diagramming notation for relational tables and constraints
  - graphically represents relationships between tables (sets of entities)
  - used for conceptual design
- We focus on Relational model
  - relations are represented as parameterized statements (“tuples”, or “predicates”)
  - used for logical design
- Other models:
  - object-oriented model
  - semi-structured data models, NoSQL (MongoDB -- `www.mongodb.org`)
  - XML
    - most relational systems can export XML interfaces
    - can provide XML storage/retrieval

Conceptual Design:

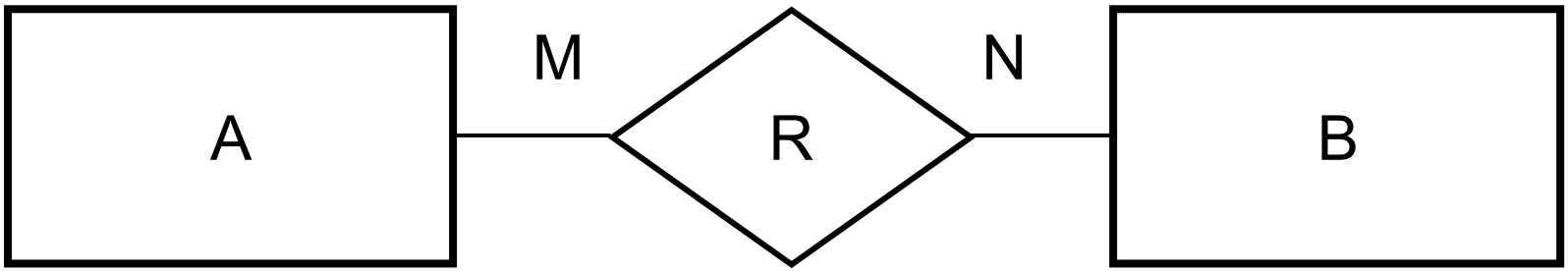
Entity Relationship Model (1)

- E-R model of real world
  - Entities (objects)
    - E.g. persons, apartments, buildings
  - Relationships between entities
    - E.g. Apartment #101 is rented by person "John Doe"
    - Or formally: Renting (John Doe, Apartment#101)
    - Relationship set "Renting" associates persons with apartments
  - Integrity constraints or business rules that hold
- Used for database conceptual design
  - Database design in E-R model usually converted to design in the ***relational model*** (described later) which is used for storage and processing

Conceptual Design:

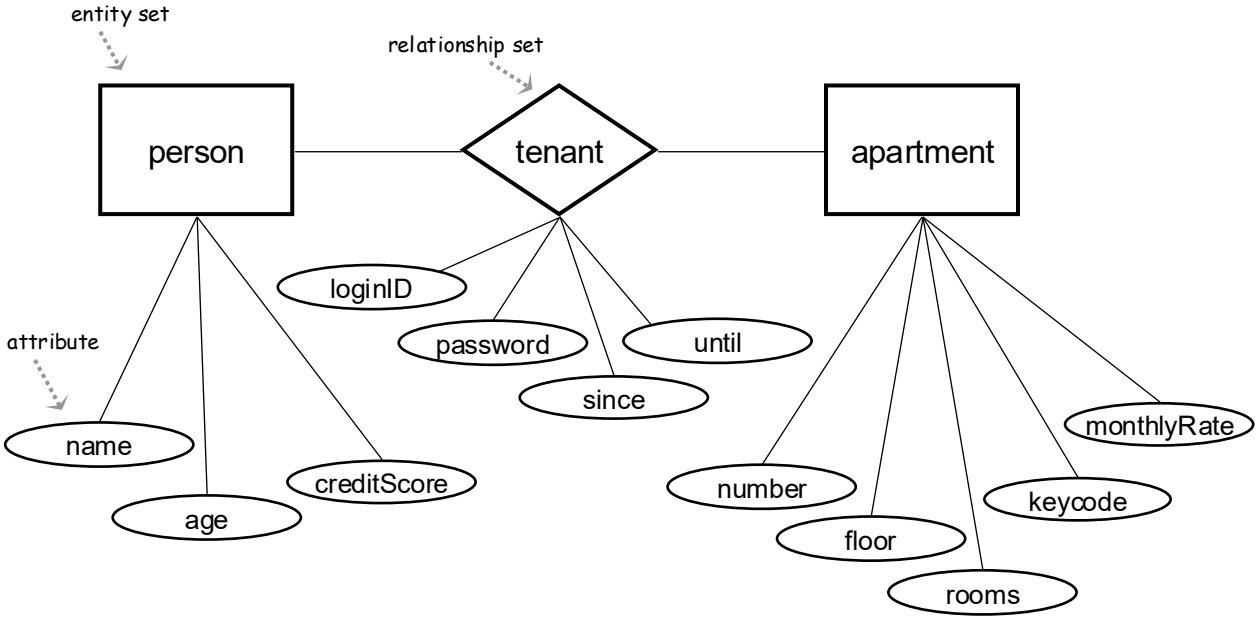
Entity Relationship Model (2)

- One-to-one: M,N = 1
- One-to-many: M = 1, N ≥ 2
- Many-to-many

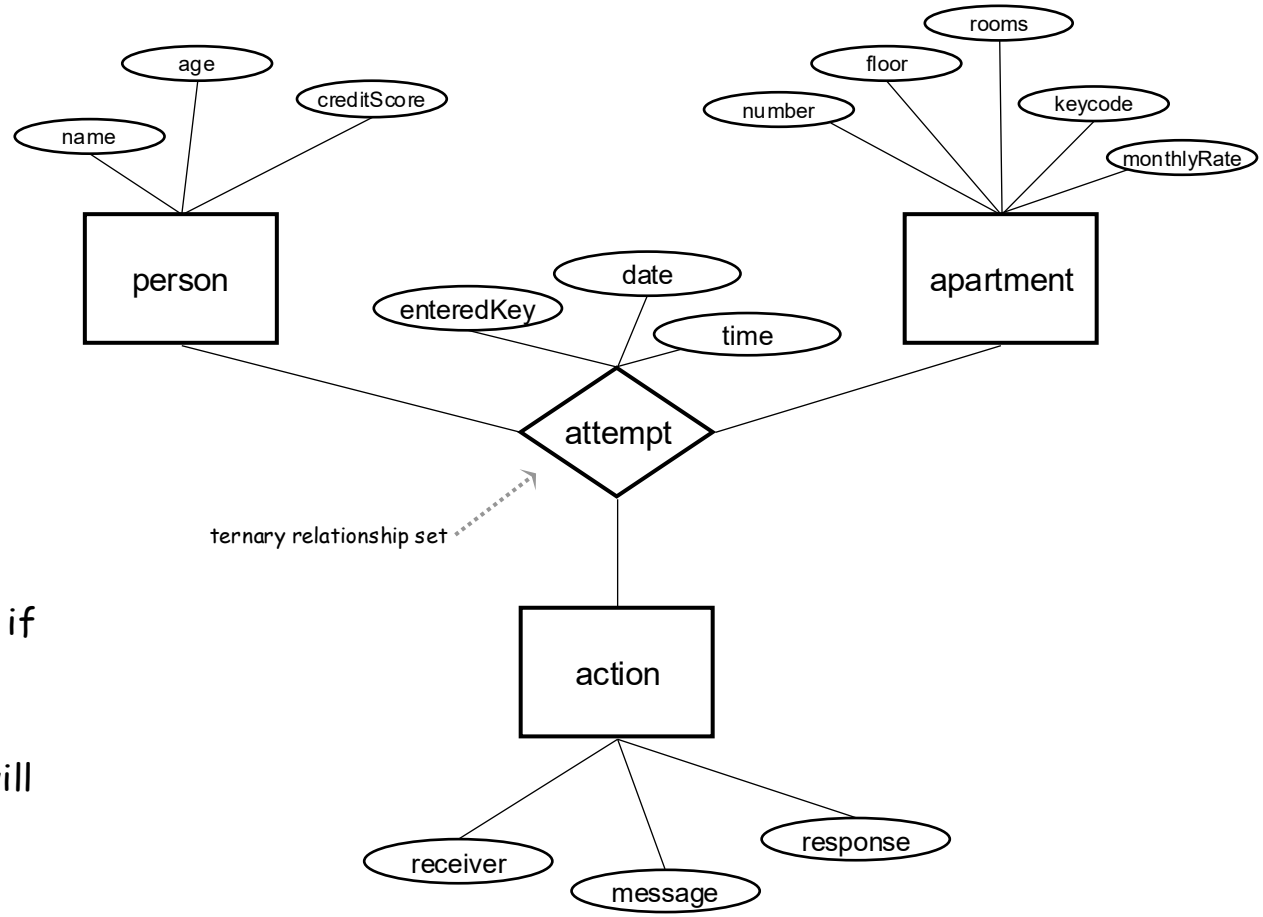


Conceptual Design:

Entity Relationship Model (3)



Conceptual Design:
Entity Relationship Model (4)

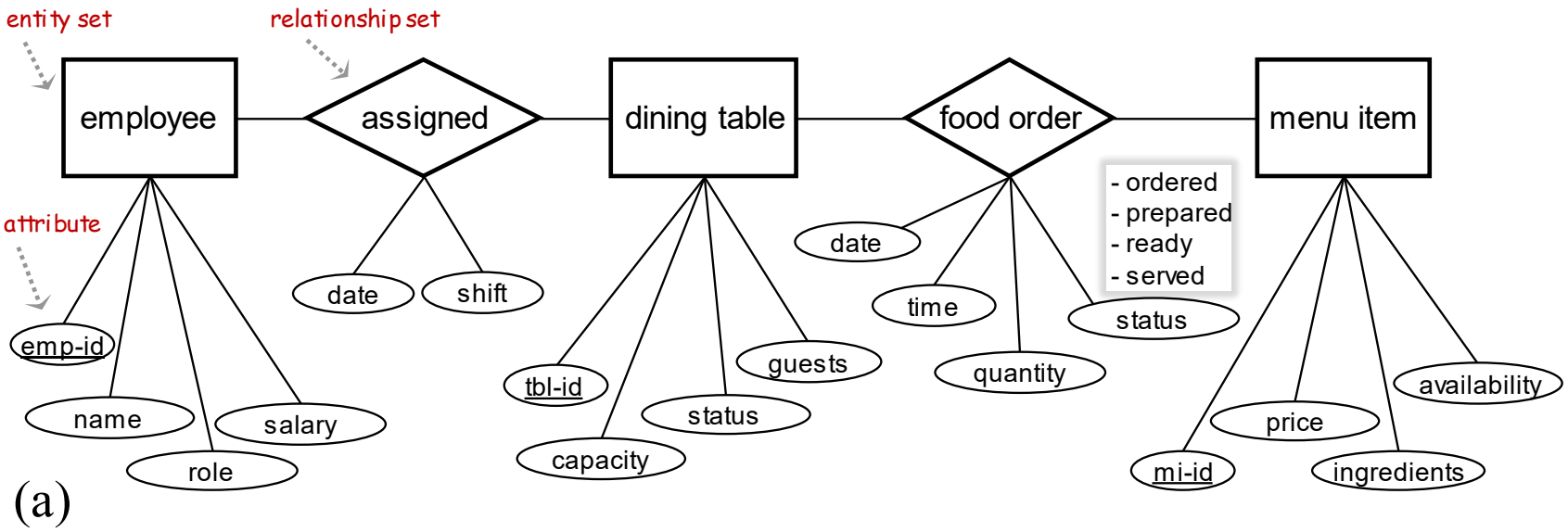


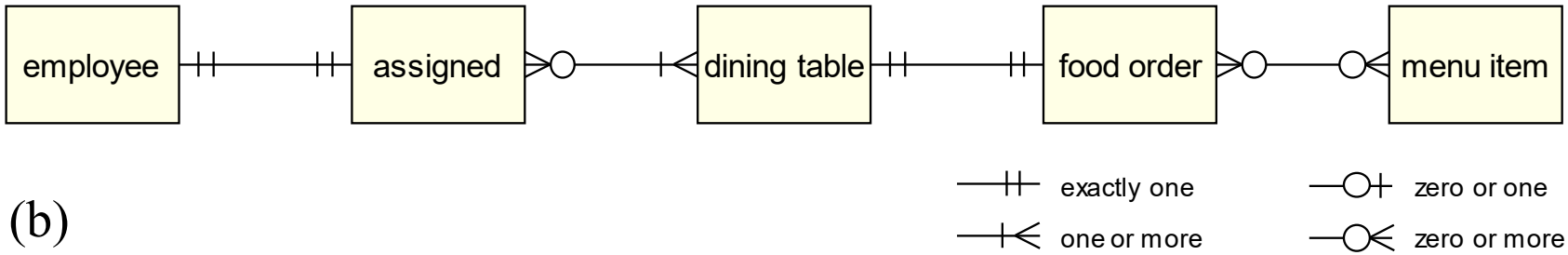
- "attempt" is an audit trail log;
- person who attempted to Unlock can be identified only if his/her "keycode" is recognized;
- otherwise, the "attempt" will be associated with a NULL (for unidentified "person");
- "action" is taken only if max allowed number of attempts is exceeded

Entity Relationship Model
— Restaurant

employee (name, role, salary, …)
assigned (date, working shift)
dining table (identifier, capacity, status, seated guests)
food order (time, item, quantity, total price, status)
menu item (id, price, ingredients, availability)
payment (time, date, amount, confirmation number)

Entity Relationship Model — Restaurant



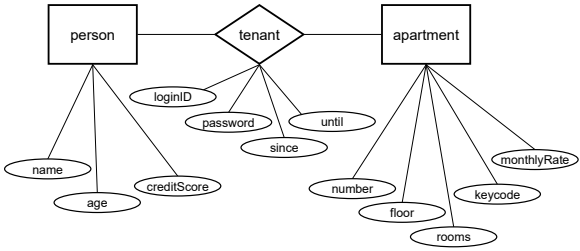


Relational Database

- Relational database: A set of "relations"
- A relation consists of 2 parts:
  - ***Schema***: specifies name of relation, plus name and datatype of each column, e.g.:
    - Tenant(loginID: string, name: string, password: string, since: date, until: date)
    - Apartment(number: integer, floor: integer, rooms: integer, keycode: integer, rate: real) … address? —no composite data!
  - ***Instance***: a table, with rows and columns
    - #rows = cardinality
    - #fields = degree / arity
- Think of a relation as a set of rows or tuples
  - "set" == all rows are distinct instances (no duplicates)

Relational Model

- Entities and Relationships in the **E-R Model** are represented as relations (tabular data) in the **Relational Model**
- Relation: Person(Identifier, Name, Age, CreditScore)
  - i.e., attributes Identifier, Name, …, are in relation Person
- Table = a set of tuples (i.e., rows)
  - Like a list…
  - …but it is unordered: no methods `first()`, no `next()`, no `last()`.
  - Rows (tuples, or records) — a tuple is an ordered set of attribute values
  - Columns (attributes)
    - Restriction: all attributes are of atomic type



Person:

attribute names (or, fields)

table name

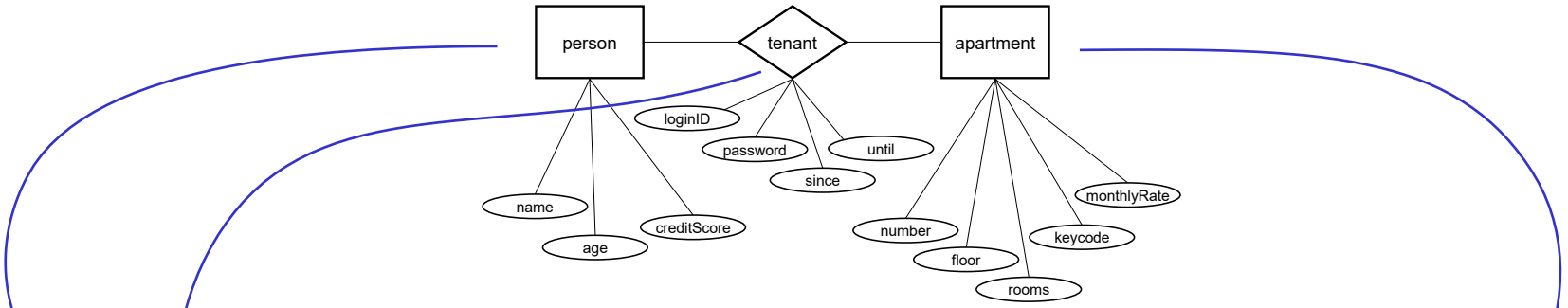
Identifier	Name	Age	CreditScore
192-83-2817	John Doer	21	690
105-04-9541	Jane Deere	21	765
429-43-1008	Bart Simpson	18	597
332-92-0006	Homer Simpson	50	620
691-55-2341	Marge Simpson	48	710

tuples / records

Relational Model — Summary

- Data Model — a way to organize information
- Schema — one particular organization,
  - i.e., a set of fields/columns, each of a given type
- Relation:
  - a name
  - a schema
  - a set of tuples/rows, each following organization specified in the schema

Mapping E-R Model to Relational Model



- Entities and Relationships to Relations:
  - Person(Identifier, Name, Age, CreditScore)
  - Apartment(Number, Rooms, KeyCode, MonthlyRate)
  - Tenant(Person.Identifier, Apartment.Number, LoginID, Password, Since, Until)

Primary key: Identifier

Primary key: Number

Foreign keys: Person.Identifier, Apartment.Number

- A **Primary Key** is an attribute selected so that it *uniquely* identifies each tuple of the relation
- A **Foreign Key** is a field whose values are keys in another relation
- Cross-reference table for many-to-many relationships (e.g., Tenant)

Structured Query Language (SQL)

- Atomic types, a.k.a. data types
- Tables built using *atomic* types
  - No composite types!
- Unlike XML, no nested tables, only flat tables are allowed!
  - We will see later how to decompose complex structures into multiple flat tables
- Query = Declarative data retrieval
  - describes ***what*** data, not ***how*** to retrieve it
  - Example: Give me the persons with credit-score > 600
    (`select * from person where creditScore > 600`)

  vs.
  - Scan the Person file one-by-one entry;
    compare each person's credit-score to 600;
    print out the entries with credit-score > 600;

Data Types in SQL

- Character strings:
  - CHAR(n) -- string, fixed length 'n' (any value from 0 to 255)
  - VARCHAR(n) -- string, variable length, maximum length 'n'

a value from 0 to 65,535; depends on vendor

- Numbers (exact and approximate):

exact:
  - BIGINT, INT, SMALLINT, TINYINT
  - MONEY -- monetary or currency values (symbol + number: $20.8)

approximate:
  - REAL, FLOAT(n) -- differ in precision

real is float(24); double precision is float(53)

- Dates and times :
  - DATE -- default format: YYYY-MM-DD
  - DATETIME -- default value: 1900-01-01 00:00:00
  - TIME -- default format: hh:mm:ss[.nnnnnnn]
- Other types... All are simple / atomic

SQL Tables

- The **schema** of a table is the table name and its attributes:

  Person(Identifier, Name, Age, CreditScore)

- A **key** is an attribute whose values are *unique*
  (ensures that table is a *set*, not a *bag*)
  we underline a key for convenience:

  Person(<u>Identifier</u>, Name, Age, CreditScore)

SQL Statements (or Commands)

- **CREATE** TABLE <table-name>
  (<field-name-1> <domain>, ...);
- **INSERT** INTO <table-name>
  (<field-name-1>, <field-name-2>, ...)
  VALUES (<field-value-1>, <field-value-2>, ...);
- **DELETE** FROM <table-name> WHERE <condition>;
- **UPDATE** <table-name>
  SET <field-name> = <value>
  WHERE <condition>;
- **SELECT** (<field-name-1>, <field-name-2>, ...)
  FROM <table-name> WHERE <condition>;
- Notes:
  - SQL Keywords are not case sensitive, but table names and column names may be
  - SQL statements can be spread over several lines
  - Single quotations (apostrophes) delimit string character values
  - Powerful variants of these statements are available

Creating Relations in SQL (1)

CREATE TABLE statement

- Creates the Person relation.
  - Note: the type (domain) of each field is specified, and enforced by the DBMS whenever tuples are added or modified.
-
```
CREATE TABLE Person
    (Identifier CHAR(11) NOT NULL UNIQUE,
    Name VARCHAR(50),
    Age INTEGER,
    CreditScore INTEGER,
    PRIMARY KEY (Identifier));
```

- It is possible to have many *candidate keys* specified using UNIQUE), one of which is chosen as the *primary key*.

SQL Domains (of valid data values)

- An **SQL Domain** is a named, user-defined set of valid data values.
  - In the sense of the domain of a function, as the set of parameter values ("inputs") for which the function is defined
  - A Schema may contain zero or more *Domains*.
  - The Objects that may belong to a Domain are known as *Domain Constraints*.
- A Domain is defined by a descriptor that contains six pieces of information:
  - name
  - data type
  - character set
  - whether reference values must be checked
  - default value (if any)
  - descriptors for domain constraints
- Advantages:
  - Using domain definitions makes it easier to see which columns are related
  - Changing a domain definition in one place changes it consistently everywhere it is used
  - Default values can be defined for domains
  - Constraints can be defined for domains

See later slides for SQL syntax ...

Creating SQL Domains

CREATE DOMAIN statement

- The CREATE DOMAIN statement names a new Domain and defines the Domain's set of valid data values
- A domain can be defined as follows:
  - `CREATE DOMAIN APT_NUM CHAR(3);  -- apartment number`
  - `CREATE DOMAIN KEY_CODE CHAR(4)  -- door key-code`

Constraints on valid values

```
CONSTRAINT constraint_1
CHECK (VALUE IS NOT NULL) NOT DEFERRABLE
CONSTRAINT constraint_2
CHECK (VALUE BETWEEN 1000 AND 9999)
DEFERRABLE INITIALLY IMMEDIATE;
```

  - The optional <Domain Constraint> list clause shows the rules that restrict the Domain's set of valid values

Creating Relations in SQL (2)

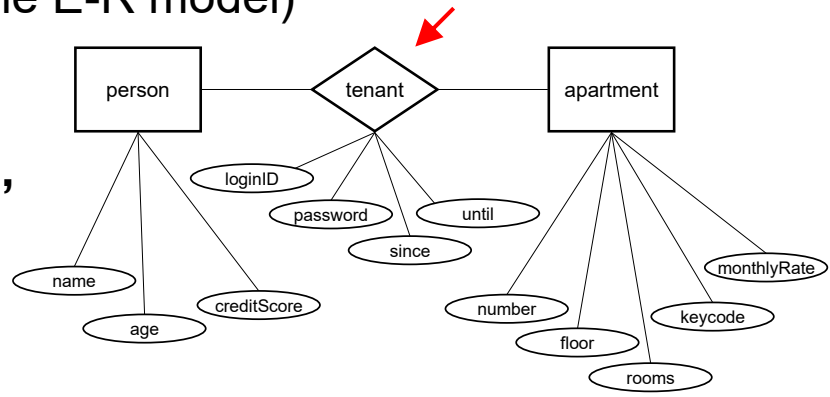
CREATE TABLE statement

- ```
  CREATE TABLE Apartment
      (Number APT_NUM NOT NULL UNIQUE,
      Rooms INTEGER,
      KeyCode KEY_CODE,
      MonthlyRate MONEY,
      PRIMARY KEY (Number));
  ```
- To add a column to a table:
  ```
  ALTER TABLE Apartment
  ADD Floor INTEGER;
  ```
- If no DEFAULT is specified, the newly added column will have NULL values for all tuples already in the database
  ```

Creating Relations in SQL (3)

CREATE TABLE statement

- **Cross-reference table** ("Relationship" in the E-R model)




```
CREATE TABLE Tenant
    (TenantID CHAR(11) NOT NULL,
    AptNum APT_NUM NOT NULL,
    LoginID VARCHAR(20),
    Password VARCHAR(20),
    Since DATE, Until DATE,
    CONSTRAINT fk_tenantID FOREIGN KEY (TenantID)
          REFERENCES Person(Identifier),
    CONSTRAINT fk_aptNum FOREIGN KEY (AptNum)
          REFERENCES Apartment(Number));
```

- Last four lines specify two `FOREIGN KEY` *constraints*
- A `FOREIGN KEY` in one table points to a `PRIMARY KEY` in another table
- Cross-reference tables do not need and do not have primary keys
- It is a good idea to *encrypt* the Password field (see a later slide)

Adding and Deleting Tuples

- Insert a single tuple using:

```
INSERT INTO Person
(Identifier, Name, Age, CreditScore)
VALUES ('192-83-2817', 'John Doer', 21, 690);
```

  - Specifying the column names (the second line above) is optional, **but watch the order of the values**!
  - Single quotations (apostrophes) delimit strings; not numbers

- Delete all tuples satisfying some condition (e.g., Name = Homer Simpson):

```
DELETE FROM Person P -- alias definition
    WHERE P.Name = 'Homer Simpson';
```

  - Aliases reduce the amount of code required for a query, and make queries simpler to understand

SQL Queries

SELECT statement

- Format:

```
SELECT A1, A2, ... An
FROM   R1, R2, ... Rm
WHERE  P;
```

- The SELECT clause specifies the attributes A_i (columns) of the result
- The FROM clause specifies the tables R_j to be scanned in the query
- The WHERE clause specifies the condition P on the columns of the tables in the FROM clause
  - It restricts which rows will appear in the result set
- Use SELECT DISTINCT to remove duplicates from the result

Simple SQL Query (1)

Person

Identifier	Name	Age	CreditScore
192-83-2817	John Doer	21	690
105-04-9541	Jane Deere	21	765
429-43-1008	Bart Simpson	18	597
332-92-0006	Homer Simpson	50	620
691-55-2341	Marge Simpson	48	710

```
SELECT *
FROM   Person
WHERE  Age >= 40;
```

Identifier	Name	Age	CreditScore
332-92-0006	Homer Simpson	50	620
691-55-2341	Marge Simpson	48	710

Simple SQL Query (2)

Person

Identifier	Name	Age	CreditScore
192-83-2817	John Doer	21	690
105-04-9541	Jane Deere	21	765
429-43-1008	Bart Simpson	18	597
332-92-0006	Homer Simpson	50	620
691-55-2341	Marge Simpson	48	710

```
SELECT Name, CreditScore
FROM   Person
WHERE  CreditScore < 650;
```

Name	CreditScore
Bart Simpson	597
Homer Simpson	620

"selection" and "projection"

Selections

What goes in the `WHERE` clause:

- x = y, x < y, x <= y, etc.
  - For number, they have the usual meanings
  - For CHAR and VARCHAR: lexicographic ordering
    - Expected conversion between CHAR and VARCHAR
  - For dates and times – just what you would expect...
- Pattern matching on strings... (next slide)

Pattern Matching on Strings:

The LIKE Operator

- s LIKE p: pattern matching on strings
- 'p' may contain two special symbols:
  - % = any sequence of characters
  - _ = any single character

Example #1:

"_ _ _%" matches any string with at least three characters

Example #2: Person(Identifier, Name, Age, CreditScore)

Find all persons whose name mentions 'D', followed by any one character, followed by 'e' :

```
SELECT *
FROM   Person
WHERE  Name LIKE '%D_e%';
```

Identifier	Name	Age	CreditScore
192-83-2817	John Doer	21	690
105-04-9541	Jane Deere	21	765

Ordering the Results

```
SELECT Name, Age, CreditScore
FROM   Person
WHERE  CreditScore > 600 AND Age < 50
ORDER BY Age, Name;
```

Person

Identifier	Name	Age	CreditScore
192-83-2817	John Doer	21	690
105-04-9541	Jane Deere	21	765
429-43-1008	Bart Simpson	18	597
332-92-0006	Homer Simpson	50	620
691-55-2341	Marge Simpson	48	710

Name	Age	CreditScore
Jane Deere	21	765
John Doer	21	690
Marge Simpson	48	710

- Ordering is *ascending* (ASC), unless you specify the DESC keyword for *descending* order: ORDER BY attribute DESC.
- Ties are broken by the second attribute on the ORDER BY list, or the third attribute, etc.

Built-in Functions

- Counting (COUNT), summation (SUM), average (AVG), minimum (MIN), maximum (MAX)
- Example: Count persons aged 21 from table Person

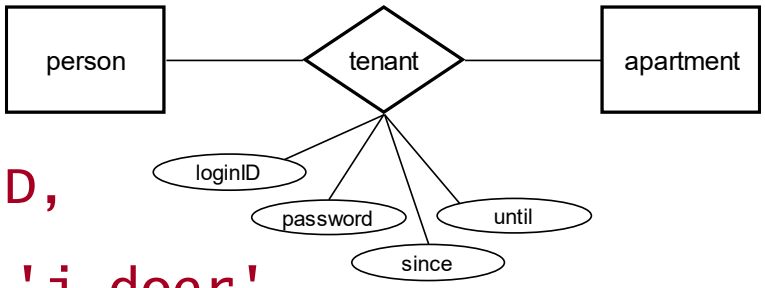
```
SELECT COUNT(*)
FROM Person
WHERE Age = 21;  -- result: 2
```

- Example: Find the average credit score by age from table Person

```
SELECT Age, AVG(CreditScore)
FROM Person
GROUP BY Age;
```

Storing Passwords in SQL

- To **encrypt** secret *password fields*, use the built-in functions `MD5()` or `SHA1()`
  - Note: SHA is an alias for SHA1




```
INSERT INTO Tenant
    (TenantID, AptNum, LoginID,
    Password, Since, Until)
 VALUES ('192-83-2817', 101, 'j.doer',
    SHA1('secretpassword'),
    2024-12-04, 2025-11-30));
```

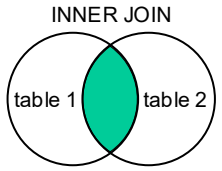
- To authenticate a tenant (e.g., during login):

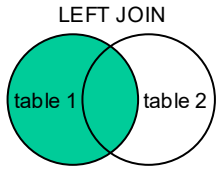
```
SELECT * FROM Tenant
    WHERE LoginID='j.doer' AND
    Password=SHA1('secretpassword');
```

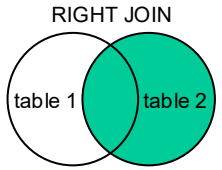
- Note: We could have encrypted also the field KeyCode in the table Apartment
- See also how to implement AES (Advanced Encryption Standard) encryption

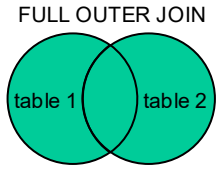
SQL Joins

- An SQL **`JOIN`** clause is used to combine rows from two or more tables, based on a common field between them
  - It creates a set of tuples that can be saved as a table or used as it is
- Standard SQL specifies five types of `JOIN`:
  - `CROSS JOIN` returns the Cartesian product of rows from tables in the join
  - `INNER JOIN` returns combined column values of two tables based on the *join condition* (predicate)
    - First takes the Cartesian product (or `CROSS JOIN`) of the two tables and then returns all rows which satisfy the join condition
  - `LEFT OUTER JOIN` (or `LEFT JOIN`) returns all rows from the first/left table, and the matched rows from the second/right table (i.e., preserves unmatched rows from the left table; fills in nulls as needed)
  - `RIGHT OUTER JOIN` (or `RIGHT JOIN`) returns all rows from the right table, and the matched rows from the left table
  - `FULL OUTER JOIN` (or `FULL JOIN`) returns all rows when there is a match in ONE of the tables
- Then there is also `NATURAL JOIN` operation
  - Specifies an inner or outer join between two tables. It has no explicit join condition. Instead, the join condition is created implicitly using the common columns (identically named) from the two tables
  - Check whether common columns exist in both tables before doing a natural join
- In MySQL, `JOIN`, `CROSS JOIN`, and `INNER JOIN` are syntactic equivalents (they can replace each other).
  In standard SQL, they are *not equivalent*. `INNER JOIN` is used with an `ON` clause, `CROSS JOIN` is used otherwise.









SQL Joins: CROSS JOIN (1)

- `CROSS JOIN` produces rows which combine each row from the first table with each row from the second table
  - The size of the result set is the number of rows in the first table multiplied by the number of rows in the second table
  - If the first table has 3 rows and 2 columns, and the second table has 2 rows and 4 columns, the result will be a table with 3×2 rows and 2+4 columns
- Example of an ***explicit*** *cross join*:

```
SELECT *
FROM Table1 CROSS JOIN Table2;
```

- Example of an ***implicit*** *cross join*:

```
SELECT *
FROM Table1, Table2;
```

Table1

a_1	a_2
ABC	123
XZ	45
A13	NULL

Table2

b_1	b_2	b_3	b_4
123	CAB	11	TUT
45	DAB	7	ANK

CROSS JOIN

result:

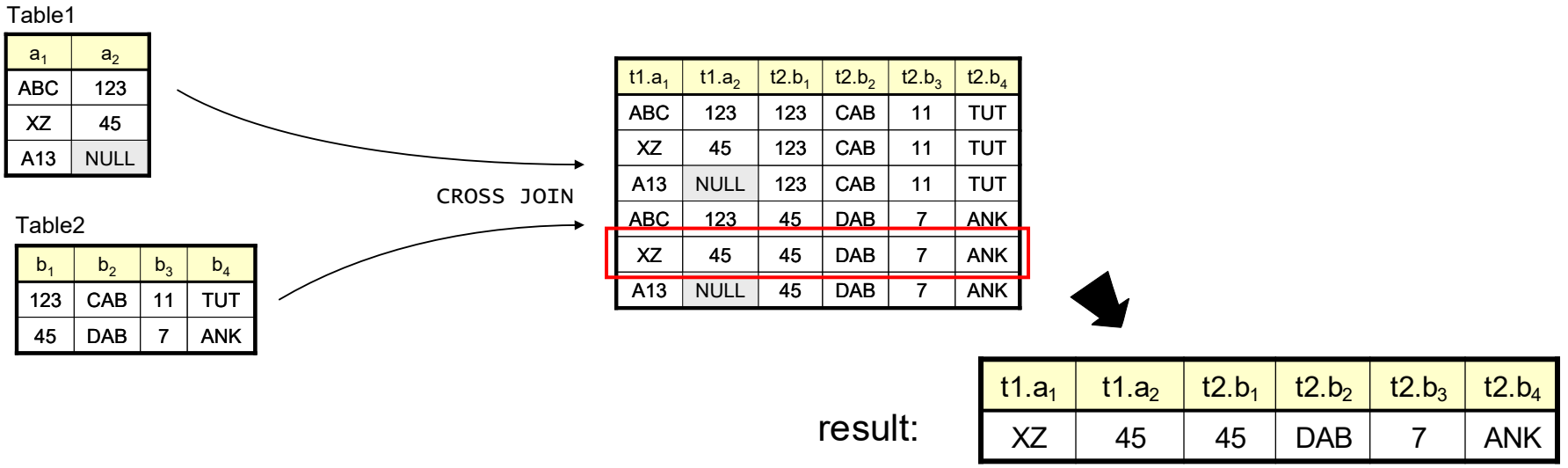
Table1 fields		Table2 fields			
$t1.a_1$	$t1.a_2$	$t2.b_1$	$t2.b_2$	$t2.b_3$	$t2.b_4$
ABC	123	123	CAB	11	TUT
XZ	45	123	CAB	11	TUT
A13	NULL	123	CAB	11	TUT
ABC	123	45	DAB	7	ANK
XZ	45	45	DAB	7	ANK
A13	NULL	45	DAB	7	ANK

SQL Joins: CROSS JOIN (2)

- A WHERE clause may be used to supply join criteria:

```
SELECT *
   FROM Table1 t1, Table2 t2
   WHERE t1.a1='XZ' AND t1.a2=t2.b1;
```

- (Note the implicit *cross join* and aliasing of table names)



SQL Joins — Example (1)

Person

Identifier	Name	Age	CreditScore
192-83-2817	John Doer	21	690
105-04-9541	Jane Deere	21	765
429-43-1008	Bart Simpson	18	597
332-92-0006	Homer Simpson	50	620
691-55-2341	Marge Simpson	48	710

Apartment

Number	Rooms	KeyCode	MonthlyRate
101	1	2021	550
102	1	1010	500
103	1	4850	600
201	2	5005	800
202	2	9083	850

Tenant

Cross-reference table:

TenantID	AptNum	LoginID	Password	Since	Until
192-83-2817	101	j.doer	secretpwd	2024-12-04	2025-11-30
105-04-9541	103	janedeere	anypwd	2025-01-15	2026-01-31
429-43-1008	202	bartules	hasnone	2020-01-01	2025-12-31
332-92-0006	201	homers	password	2020-01-01	2025-12-31
691-55-2341	201	margesim	123457	2020-01-01	2025-12-31

- Example join query: Find monthly rates for apartments where tenants have credit score greater than 700

SQL Joins — Example (2)

Query: "Find monthly rates for apartments where tenants have credit score greater than 700"

- We need information from two tables: `Person` and `Apartment`
- First, perform a *cross join* of these tables using a `SELECT` statement that has the tables named in the `FROM` clause
- Second, form the `WHERE` clause to list these three conditions:
  - The `CreditScore` column of the `Person` table must be greater than 700
  - The `Identifier` column of the `Person` table must match the `TenantID` column of the `Tenant` table
  - The `AptNum` column of the table `Tenant` must match the `Number` column of the `Apartment` table
- The SQL code is shown next ...

SQL Joins — Example (3)

Query: “Find monthly rates for apartments where tenants have credit score greater than 700”

- We need information from two tables

```
SELECT apt.Number, apt.MonthlyRate
  FROM Person p, Apartment apt
  WHERE p.CreditScore >= 700 AND
      p.Identifier = Tenant.TenantID AND
          Tenant.AptNum = apt.Number;
```

- Result set:

apt.Number	apt.MonthlyRate
103	600
201	800

SQL Transactions

- Transaction is a single unit of work.
  - If successful, all data modifications guaranteed to be committed
  - If errors encountered, cancel or roll back

```
Start transaction;
### modify database ###
Commit;
### encounter errors ###
Rollback;
```